Payment Service

Technical Documentation

Technology Stack: Kotlin, Ktor, Koin, Restate, TigerBeetle, PostgreSQL, Exposed, HikariCP

V 1.0.0

By Joshua Ochayon

1. System Overview

The Payment Service is a Kotlin-based financial application responsible for managing customers, merchants, systems, accounts, payment configuration, payments, and ledger transfers.

The service separates application state from financial ledger state:

- PostgreSQL stores application entities, account metadata, fee configuration, and payment records.
- TigerBeetle is the financial ledger responsible for account balances and financial transfers.
- Restate provides durable execution of the payment workflow, including retry handling and terminal workflow failures.
- Ktor exposes the HTTP API.
- Koin provides dependency injection and application composition.

The central design principle is that **PostgreSQL records what the application knows about a payment,** while **TigerBeetle records the actual movement of funds**. **Restate coordinates the operations** required to move a payment through its lifecycle.

---

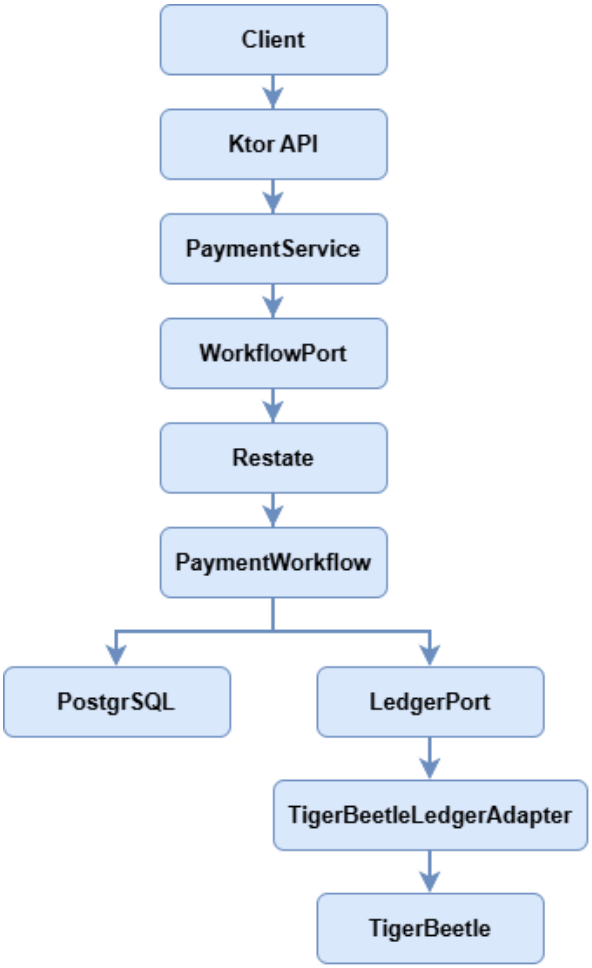
2. Architecture

The application follows a layered architecture with explicit boundaries between HTTP handling, application services, domain concepts, and infrastructure.

This separation means that the payment workflow does not directly depend on a concrete TigerBeetle implementation or directly expose database implementation details to the API.

The domain defines ports such as LedgerPort and WorkflowPort, while infrastructure supplies their implementations.

The architecture separates application state, durable workflow execution, and financial ledger state so that each system has a clear responsibility.



Architectural Source of Truth

- PostgreSQL is the source of truth for application entities and payment business state.
- TigerBeetle is the authoritative financial ledger for balances and financial transfers.
- Restate provides durable execution, workflow coordination, and retry semantics.

These responsibilities are intentionally separate. PostgreSQL application records and TigerBeetle ledger transfers describe different aspects of the same business process and should not be treated as interchangeable sources of truth.

---

3. Project Structure

api/
- *routes
- request/
- response/
- dto/

application/
- *services

domain/
- *entities
- enums
- ledger/
- port/

infrastructure/

database/
- DatabaseFactory
- DatabaseConfig
- DatabaseSeeder
- repository/
- table/

ledger/
- TigerBeetleLedgerAdapter
- LedgerSeeder

workflow/
- PaymentWorkflow
- RestateModule
- RestateWorkflowAdapter

---

4. Component Responsibilities

Ktor	HTTP server and API boundary.
Koin	Dependency injection and component wiring.
Restate	Durable workflow execution and retry coordination.
PostgreSQL	Application entities and payment business state.
Exposed	Kotlin database access and SQL mapping.
HikariCP	PostgreSQL connection pooling.
TigerBeetle	authoritative financial ledger and balance management.

---

5. Layer Responsibilities

API Layer

Exposes HTTP endpoints, parses and validates transport input, delegates work to application services, and maps application exceptions to HTTP responses. It does not contain core payment orchestration.

Application Layer

Coordinates use cases and depend on domain ports rather than concrete infrastructure implementations.

Domain Layer

Contains business entities, business concepts, value types, and ports that define required infrastructure capabilities.

Infrastructure Layer

Provides concrete implementations for database persistence, TigerBeetle ledger operations, and Restate workflow execution.

This follows a ports-and-adapters approach: application and domain logic depend on abstractions, while infrastructure implementations sit at the boundary.

---

6. Domain Model

The domain models the business concepts required to process payments: owners, accounts, assets, fee configuration, payments, and ledger operations.

---

7. Payment Workflow

The payment workflow is the central orchestration component. Restate provides the durable execution boundary.

The workflow is responsible for coordinating:

- Validation
- Fee calculation
- Payment persistence
- Ledger execution
- Payment status

Workflow sequence

A. Generate durable payment identity

A payment ID is generated inside a durable workflow block.

B. Generate durable timestamp

The payment creation timestamp is also generated as a durable value.

C. Validate amount

The amount must be greater than zero.

Otherwise:

INVALID_TRANSACTION_AMOUNT

D. Validate accounts are different

The debtor and creditor cannot be the same account.

Otherwise:

INVALID_ACCOUNTS

E. Validate debtor account

The debtor must exist.

Otherwise:

DEBTOR_ACCOUNT_NOT_FOUND

F. Validate creditor account

The creditor must exist.

Otherwise:

CREDITOR_ACCOUNT_NOT_FOUND

G. Validate account status

Payment accounts must be ACTIVE.

Other statuses are rejected, such as:

DEBTOR_ACCOUNT_NOT_ACTIVE

CREDITOR_ACCOUNT_NOT_ACTIVE

H. Retrieve fee configuration

The workflow retrieves the fee configuration for:

PaymentType + Asset

If no configuration exists:

FEE_CONFIGURATION_NOT_FOUND

I. Calculate fee

The configured rate is applied to the payment amount.

J. Validate available funds

The available balance must cover payment amount + fee amount.

The application validates that the available balance covers the payment amount plus fee. Available funds are authoritatively validated by TigerBeetle when the transfer batch is submitted. If the debit exceeds the permitted balance, the result is mapped to InsufficientFundsException (INSUFFICIENT_FUNDS)

K. Generate transfer references

A payment reference is generated.

A fee reference is generated only when a fee transfer is required.

L. Create PENDING payment

The application payment record is persisted as:

PENDING

before the financial ledger operation.

M. Create ledger transfers

The payment and optional fee transfer are submitted to TigerBeetle.

N. Handle ledger result

Expected business failures result in:

PENDING → REJECTED

Successful execution results in:

PENDING → SUCCESS

O. Return references

The workflow returns:

paymentReference

feeReference

The fee reference is null when no fee transfer exists.

---

8. Payment State Machine

PENDING

The payment has been accepted into the durable workflow and recorded in PostgreSQL, but the financial operation has not yet completed successfully.

SUCCESS

The required TigerBeetle transfer operation completed successfully.

REJECTED

The payment was rejected due to an expected business condition.

Examples:

- Insufficient funds
- Missing debtor
- Missing creditor
- Invalid business state

REVERSED

REVERSED represents a future/extended lifecycle state for a payment that was subsequently reversed.

---

9. Restate Execution and Retry Semantics

Restate is responsible for durable workflow execution.

The payment workflow uses runBlock around operations that must remain stable across workflow replay.

Terminal failures

Some errors are known business outcomes and should not be retried.

These include:

- INVALID_TRANSACTION_AMOUNT
- INVALID_ACCOUNTS
- ACCOUNT_NOT_ACTIVE
- DEBTOR_ACCOUNT_NOT_FOUND
- CREDITOR_ACCOUNT_NOT_FOUND
- INSUFFICIENT_FUNDS
- FEE_CONFIGURATION_NOT_FOUND

The workflow throws TerminalException for these cases. The resulting error is translated back to the API layer as a meaningful payment error.

---

10. TigerBeetle / Ledger Design

TigerBeetle is the authoritative financial ledger.

The application uses a dedicated adapter – this provides the hexagonal/ports-and-adapters design separating the Ledger from the business logic and keeps TigerBeetle-specific types and API calls out of the rest of the application:

LedgerPort

↓

TigerBeetleLedgerAdapter

↓

TigerBeetle Client

Account IDs

Application account numbers are converted to TigerBeetle UInt128 IDs.

Transfer IDs

Application payment and fee references are UUIDs.

They are converted into TigerBeetle UInt128 transfer IDs.

Therefore, the same reference can be used to correlate:

Application payment

↕

TigerBeetle transfer

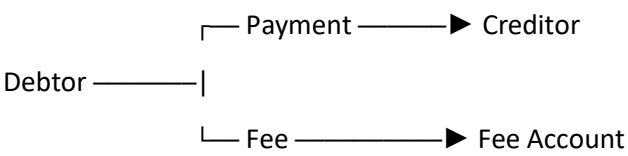
Linked transfers

When a payment has a fee, the workflow creates:

Payment transfer

Fee transfer

and submits them together as a linked batch.



If the fee amount is zero, only the payment transfer is submitted.

The adapter dynamically sizes the TigerBeetle TransferBatch based on the number of requested transfers.

Ledger result mapping

TigerBeetle statuses are translated into domain exceptions:

- CreateTransferStatus.Created → success
- CreateTransferStatus.Exists → success
- CreateTransferStatus.ExceedsCredits → InsufficientFundsException
- CreateTransferStatus.DebitAccountNotFound → DebtorAccountNotFoundException
- CreateTransferStatus.CreditAccountNotFound → CreditorAccountNotFoundException
- Other → LedgerOperationException

This prevents TigerBeetle implementation details from leaking into the API.

---

11. Database Design

PostgreSQL is the application's persistence layer.

The major tables are:

- CustomerTable
- MerchantTable
- SystemTable
- AccountTable
- PaymentTable
- FeeType

The database is accessed through repository classes.

Fee configuration

Fee configuration is selected using:

- paymentType
- asset

The database enforces uniqueness so there is one applicable configuration for each combination.

Database transactions

Repository operations execute inside Exposed transactions.

The database connection pool is managed by HikariCP.

---

12. Configuration

The current development configuration is:

Component	Host-visible port
Ktor API	8080
Payment workflow endpoint	9090
Restate UI/Admin	9070
Restate ingress	9072
PostgreSQL	5433
TigerBeetle	3000

The application binds to 0.0.0.0 so it can accept connections on all network interfaces available to the process or container. Clients in local development access the service through localhost rather than using 0.0.0.0 as a destination.

Docker services communicate over the payment-network bridge. Service names are used for normal container-to-container discovery, while the configured TigerBeetle address provides the required stable network endpoint.

---

13. Development Seed Data

V1 includes development seeders for both PostgreSQL and TigerBeetle.

These accounts provide a predictable development environment for testing payment and fee transfers:

- 1 – System settlement account
- 2 – System fee revenue account
- 1001 & 1002 – Customer transactional account
- 2001 & 2002 – Customer transactional account

Database seeding establishes the corresponding application-side development data and fee configuration.

Seeders are intended for development/testing and must be removed for a production implementation.